

Programming Foundations

Level 01 — Core Concepts & Fundamentals

From zero to programmer — a complete beginner-to-professional reference guide

Internship Module	Summer Internship — Programming & Automation (T08)
Level	Level 1 — Programming Foundations
Topics Covered	Logic · Variables · Data Types · Operators · Functions · Loops · Conditions · Error Handling · File Operations · Debugging
Prepared By	Rama Bhere & Piyush Kanbhat
Purpose	Summer Internship Technical Documentation

Concepts Covered in This Document :

01 Logic	02 Variables	03 Data Types	04 Operators	05 Functions
06 Loops	07 Conditions	08 Error Handling	09 File Operations	10 Debugging

TABLE OF CONTENTS

1.	Logic — The Foundation of Thinking in Code	3
2.	Variables — Containers for Data	5
3.	Data Types — Understanding Kinds of Information	7
4.	Operators — Performing Actions on Data	9
5.	Functions — Reusable Blocks of Code	11
6.	Loops — Repeating Tasks Automatically	13
7.	Conditions — Making Decisions in Code	15
8.	Error Handling — Dealing with Mistakes Gracefully	17
9.	File Operations — Reading and Writing Data	19
10.	Debugging — Finding and Fixing Problems	21

INTRODUCTION

Welcome to Level 1 of the T08 — Programming & Automation module. This document is a comprehensive technical reference guide prepared by Rama Bhare and Piyush Kanbhat as part of their Summer Internship Programme. The guide covers ten foundational concepts of programming that every developer — from a first-time coder to a seasoned software engineer — must understand thoroughly.

Programming is the art of giving precise instructions to a computer to solve problems. Before writing any real-world application, a programmer must master the basic building blocks: how to think logically, how to store and work with data, how to make decisions, how to repeat actions efficiently, and how to handle unexpected situations gracefully.

Each section of this document is structured to be accessible to a 5th-grade student while simultaneously providing enough depth and technical precision to satisfy an experienced professional. Code examples use Python — a language celebrated for its clean, readable syntax — but the concepts apply universally across all modern programming languages.

01. Logic — The Foundation of Thinking in Code

Logic is the backbone of every program ever written. Before any line of code runs, a programmer must think clearly about what problem they are solving and the exact steps needed to solve it. Logic is the discipline of structuring those thoughts into precise, unambiguous instructions that a computer can follow without confusion.

What is Logic in Programming?

In everyday life, we constantly use logic : "If it is raining, I will take an umbrella; otherwise, I will wear sunglasses." Computers follow this same pattern — they evaluate conditions and choose a path based on whether something is true or false. Programming logic is the formal application of this thinking process.

Key Logical Concepts

- Boolean Values — Everything in logic is ultimately True or False.
- AND — Both conditions must be true for the overall result to be true.
- OR — At least one condition must be true for the overall result to be true.
- NOT — Inverts a value; True becomes False and False becomes True.
- Truth Tables — Diagrams that show every possible combination of logical inputs and their outputs.

Logical Operators — Quick Reference Table

Operator	Meaning	Example
AND	Both must be True	True AND True → True
OR	At least one must be True	True OR False → True
NOT	Flips the value	NOT True → False
XOR	Exactly one must be True	True XOR True → False

Code Example

```
# Logic in Python
is_raining = True
has_umbrella = False

# AND operator
if is_raining and has_umbrella:
    print("Stay dry!")
elif is_raining and not has_umbrella:
    print("You will get wet!")
else:
    print("Enjoy the sunshine!")

# Output: You will get wet!
```

Why Logic Matters ?

Every decision a computer makes — from displaying a webpage to routing a self-driving car — is the result of logical operations. Writing clean, correct logic is the single most important skill a programmer can develop. Poor logic leads to bugs, crashes, and incorrect results regardless of how well the rest of the code is written.

 **Real-World Analogy :** A traffic light uses logic — if the light is red AND a car is approaching, the car must stop. Programming applies this same principle at an enormous scale.

02. Variables — Containers for Data

A variable is a named storage location in a computer's memory that holds a value. Think of it as a labelled box: you can put something in, look at what's inside, and change the contents whenever needed. Variables are the most fundamental tool for working with data in any programming language.

Declaring and Assigning Variables

In most languages, creating a variable requires giving it a name and assigning a value. The assignment operator (=) places a value into the variable.

```
# Python automatically detects the type
student_name = "Rama Bhere"    # Text (string)
age          = 20               # Whole number (integer)
gpa          = 3.85             # Decimal number (float)
is_enrolled  = True             # Yes/No value (boolean)

# Variables can be updated
age = 21                       # age is now 21
print(student_name, "is", age, "years old")
```

Rules for Naming Variables

- Must start with a letter or underscore (`_`), never a number.
- Can contain letters, digits, and underscores only.
- Case-sensitive: `myName` and `myname` are two different variables.
- Cannot be a reserved keyword (e.g., `if`, `for`, `while`, `return`).
- Use descriptive names: `student_age` is better than `x`.

Naming Conventions

Convention	Example	Common Usage
snake_case	student_name	Python variables & functions
camelCase	studentName	JavaScript, Java
PascalCase	StudentName	Class names (all languages)

Variable Scope

Scope defines where in your code a variable can be accessed. Understanding scope is critical to avoiding confusing bugs.

- Global Variable — Declared outside all functions; accessible anywhere in the program.
- Local Variable — Declared inside a function; only accessible within that function.

```
message = "Hello from global scope" # Global

def greet():
    local_msg = "I am local"      # Local
    print(message)                # OK — can access global
    print(local_msg)              # OK — local exists here

greet()
# print(local_msg) # ERROR — local_msg does not exist here
```

 **Best Practice :** Avoid overusing global variables. Pass data through function parameters to keep your code clean, predictable, and easy to test.

03. Data Types — Understanding Kinds of Information

A data type tells the computer what kind of value a variable holds and what operations can be performed on it. Knowing data types helps write accurate programs and prevents errors that arise from performing nonsensical operations, such as dividing a name by 2.

Core Data Types

Data Type	Description	Example Value	Use Case
int	Whole numbers, positive or negative	42, -7, 0	Age, count, quantity
float	Numbers with decimal points	3.14, -0.5, 2.0	Price, temperature, GPA
str	Text enclosed in quotes	"Hello", "Python"	Names, messages, URLs
bool	True or False only	True, False	Flags, conditions, switches
list	Ordered, changeable collection	[1, 2, 3], ["a", "b"]	Shopping lists, scores

dict	Key-value pairs	{"name": "Piyush"}	Records, configurations
------	-----------------	--------------------	----------------------------

Type Conversion (Casting)

Sometimes you need to convert a value from one type to another. This is called type casting or type conversion.

Type conversion examples

```
number_str = "42"          # This is a string, not a number
```

```
number_int = int(number_str) # Convert to integer → 42
```

```
numberflt = float(number_str) # Convert to float → 42.0
```

```
pi = 3.14159
```

```
pi_int = int(pi)          # → 3 (truncates decimal)
```

```
pi_str = str(pi)          # → "3.14159" (text)
```

Check the type of a variable

```
print(type(42))          # <class "int">
```

```
print(type("Hello"))     # <class "str">
```

```
print(type(True))        # <class "bool">
```

Mutable vs Immutable Types

- **Immutable** : Immutable types cannot be changed after creation: int, float, str, bool, tuple. Operations create a new object rather than modifying the original.

- **Mutable** : Mutable types can be changed in place: list, dict, set. You can add, remove, or update items without creating a new object.

💡 **Pro Tip** : Always use the most specific type for your data. Use int for counts, float for measurements, bool for flags. Choosing the right type makes your code self-documenting and prevents subtle bugs.

04. Operators — Performing Actions on Data

Operators are special symbols that perform operations on values (called operands). They are the verbs of a programming language — they describe what to do with data. Python and most languages share the same fundamental categories of operators.

1. Arithmetic Operators

```
a, b = 10, 3
print(a + b) # Addition    → 13
print(a - b) # Subtraction → 7
print(a * b) # Multiplication → 30
print(a / b) # Division    → 3.333...
print(a // b) # Floor Division → 3 (drops decimal)
print(a % b) # Modulo      → 1 (remainder)
print(a ** b) # Exponentiation → 1000 (10 to the power of 3)
```

2. Comparison Operators

Comparison operators evaluate to True or False and are used in conditions and loops.

```
x, y = 5, 10
print(x == y) # Equal to      → False
print(x != y) # Not equal to  → True
print(x > y)  # Greater than   → False
print(x < y)  # Less than      → True
```

```
print(x >= y) # Greater than or equal → False  
print(x <= y) # Less than or equal → True
```

3. Logical Operators

```
a, b = True, False  
print(a and b) # AND → False (both must be True)  
print(a or b) # OR → True (at least one must be True)  
print(not a) # NOT → False (inverts the value)
```

4. Assignment Operators

```
score = 100 # Assign  
score += 10 # score = score + 10 → 110  
score -= 5 # score = score - 5 → 105  
score *= 2 # score = score * 2 → 210  
score //= 3 # score = score // 3 → 70
```

5. Operator Precedence (Order of Operations)

Like mathematics, programming operators follow a defined order of precedence — BODMAS/PEMDAS applies. Use parentheses to make your intent explicit and avoid confusion.

```
result = 2 + 3 * 4 # → 14 (multiplication first)
```

```
result = (2 + 3) * 4    # → 20 (parentheses override)
```

```
result = 10 - 4 / 2     # → 8.0 (division first)
```

 **Common Mistake :** = is assignment (stores a value). == is comparison (checks equality). Writing `if x = 5` instead of `if x == 5` is one of the most frequent beginner errors.

05. Functions — Reusable Blocks of Code

A function is a named, self-contained block of code designed to perform a specific task. Instead of writing the same instructions over and over, you write them once inside a function and call that function whenever you need the task performed. This is the DRY principle: Don't Repeat Yourself.

Defining and Calling a Function

```
# Defining a function
```

```
def greet(name):  
    """Returns a personalised greeting message."""  
    message = f'Hello, {name}! Welcome to Programming.'  
    return message
```

```
# Calling the function
```

```
result = greet("Rama")  
print(result) # Hello, Rama! Welcome to Programming.  
  
result2 = greet("Piyush")  
print(result2) # Hello, Piyush! Welcome to Programming.
```

Anatomy of a Function

- `def` keyword — signals the start of a function definition.
- Function name — follows the same naming rules as variables (use `snake_case`).

- Parameters — inputs listed inside parentheses; a function can have zero or more.
- Docstring — an optional description inside triple quotes explaining what the function does.
- Body — the indented block of code that executes when the function is called.
- return statement — sends a value back to the caller; functions without return give back None.

Types of Function Arguments

1. Required (positional) arguments

```
def add(a, b):  
    return a + b  
  
print(add(3, 5))      # → 8
```

2. Default arguments

```
def power(base, exponent=2):  
    return base ** exponent  
  
print(power(4))        # → 16 (exponent defaults to 2)  
print(power(2, 10))    # → 1024
```

3. Keyword arguments (order does not matter)

```
print(power(exponent=3, base=2)) # → 8
```

4. Variable-length arguments

```
def total(*numbers):  
    return sum(numbers)
```

```
print(total(1, 2, 3, 4, 5)) # → 15
```

Why Functions Are Essential

- Reusability — write once, use many times across the program.
- Readability — a well-named function makes code read like plain English.
- Maintainability — fix a bug in one place and it is fixed everywhere the function is called.
- Testability — small, focused functions are easy to test and verify independently.
- Abstraction — hide complex implementation details behind a simple interface.

 **Design Principle :** A well-designed function should do exactly one thing and do it well. If you find yourself naming a function "do_this_and_that", it is a sign the function should be split into two.

06. Loops — Repeating Tasks Automatically

A loop is a programming construct that repeats a block of code multiple times. Instead of copying and pasting the same instructions ten times, you write them once and tell the loop how many times to run. Loops are among the most powerful tools in programming, enabling computers to process massive amounts of data quickly.

1. The for Loop — Iterating Over a Sequence

A for loop runs a fixed number of times — once for each item in a sequence (list, string, range, etc.).

```
# Loop over a list
```

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(f'I like {fruit}')
```

```
# Output:
```

```
# I like apple
```

```
# I like banana
```

```
# I like cherry
```

```
# Loop using range()
```

```
for i in range(1, 6): # 1 through 5 (6 is excluded)
```

```
    print(f'Count: {i}')
```

```
# Loop over a string character by character
```

```
for char in "HELLO":  
    print(char, end=" ") # H E L L O
```

2. The while Loop — Repeating Until a Condition is False

A while loop continues running as long as its condition remains True. It is ideal when you do not know in advance how many times the loop must run.

```
password = ""  
while password != "open123":  
    password = input("Enter the password: ")  
    if password != "open123":  
        print("Incorrect. Try again.")  
print("Access granted!")
```

Loop Control Statements

Statement	What it Does
break	Immediately exits the loop, skipping remaining iterations.
continue	Skips the rest of the current iteration and moves to the next one.
pass	Does nothing; used as a placeholder when a body is syntactically required.

```
# break and continue in action
for num in range(1, 11):
    if num == 6:
        break          # Stop when num reaches 6
    if num % 2 == 0:
        continue       # Skip even numbers
    print(num, end=" ") # Output: 1 3 5
```

⚠ **Infinite Loops :** A while loop whose condition never becomes False will run forever, freezing your program. Always ensure there is a clear exit condition. Use a break as a safety valve if needed.

07. Conditions — Making Decisions in Code

Conditions allow a program to choose different paths of execution based on data. Without conditions, a program would perform the exact same actions every time it runs, regardless of input or state. Conditional logic is what makes software dynamic, intelligent, and responsive.

The if / elif / else Structure

The if statement evaluates a condition. If True, the indented block executes. If False, Python checks the next elif (else if) condition. If none match, the else block runs as a default.

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
```

```
print(f'Score: {score}, Grade: {grade}') # Score: 85, Grade: B
```

Nested Conditions

Conditions can be nested inside one another to test multiple criteria in sequence.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed.")
    else:
        print("Please show your ID.")
else:
    print("Sorry, you must be 18 or older.")
```

Conditional Expressions (Ternary Operator)

Python supports a concise one-line conditional expression for simple decisions.

```
# Syntax: value_if_true if condition else value_if_false
temperature = 30
weather = "Hot" if temperature > 25 else "Cool"
print(weather) # Hot

# Equivalent long form:
# if temperature > 25:
```

```
# weather = "Hot"
# else:
# weather = "Cool"
```

Common Condition Patterns

- Check membership: `if item in my_list`
- Check emptiness: `if not my_list` (True when list is empty)
- Check type: `if isinstance(x, int)`
- Check None: `if variable is None` (never use `==` for None)

 **Readability Tip :** Complex conditions can be hard to read. Name your boolean expressions using descriptive variables: `is_admin = age > 18 and has_permission`. Then write: `if is_admin`. This dramatically improves code clarity.

08. Error Handling — Dealing with Mistakes Gracefully

No matter how carefully a program is written, unexpected things happen: a file might not exist, a user might enter letters where numbers are expected, or a network connection might drop. Error handling is the practice of anticipating these problems and responding to them gracefully, rather than letting the program crash with an unhelpful error message.

Types of Errors in Python

Error Type	Cause	Example
SyntaxError	Code violates language grammar rules	missing colon after if
TypeError	Wrong type in an operation	"5" + 5 (str + int)
ValueError	Right type, wrong value	int("hello")
ZeroDivisionError	Division by zero	10 / 0
FileNotFoundError	File does not exist	open("missing.txt")

The try / except / else / finally Structure

```
def safe_divide(a, b):  
    try:  
        result = a / b      # Code that might raise an error  
    except ZeroDivisionError:  
        print("Error: Cannot divide by zero!")  
        return None  
    except TypeError as e:  
        print(f"Type error: {e}")  
        return None  
    else:  
        print("Division successful.") # Runs ONLY if no error  
        return result  
    finally:  
        print("Execution complete.") # ALWAYS runs  
  
print(safe_divide(10, 2)) # 5.0  
print(safe_divide(10, 0)) # Error message, then None
```

Raising Custom Exceptions

```
def set_age(age):  
    if age < 0 or age > 150:  
        raise ValueError(f"Invalid age: {age}. Must be between 0 and 150.")  
    return age
```

```
try:
    set_age(-5)
except ValueError as e:
    print(e) # Invalid age: -5. Must be between 0 and 150.
```

Error Handling Best Practices

- Catch specific exceptions, not bare except: — catching everything hides bugs.
- Always log or display the error message so it can be diagnosed later.
- Use finally for cleanup code (closing files, database connections) that must always run.
- Do not use exceptions for normal flow control — they are for exceptional situations.
- Validate user input before processing it to prevent errors reaching critical code.

 **Professional Insight :** In production software, errors are always logged to a file or monitoring system. Never silently swallow an exception with an empty except block — this is one of the most dangerous anti-patterns in software engineering.

09. File Operations — Reading and Writing Data

File operations allow programs to persist data beyond a single run. Without the ability to read from and write to files, every piece of data a program creates would be lost the moment it closes. Files are the bridge between a running program and permanent storage.

File Open Modes

Mode	Meaning	Behaviour if File Missing
"r"	Read (default)	Raises <code>FileNotFoundError</code>
"w"	Write — overwrites entire file	Creates a new file
"a"	Append — adds to end without deleting	Creates a new file
"x"	Exclusive creation	Raises error if file exists
"rb"	Read in binary mode (images, PDFs, etc.)	Raises <code>FileNotFoundError</code>

Reading Files

Best practice: use "with" — automatically closes the file

with open("students.txt", "r") as file:

```
    content = file.read()      # Read entire file as string
    print(content)
```

Read line by line (memory-efficient for large files)

with open("students.txt", "r") as file:

for line in file:

```
        print(line.strip())    # .strip() removes newline character
```

Read all lines into a list

with open("students.txt", "r") as file:

```
    lines = file.readlines()   # ["Line1\n", "Line2\n", ...]
```

Writing Files

Write to a file (creates or overwrites)

with open("report.txt", "w") as file:

```
    file.write("Summer Internship Report\n")
    file.write("Author: Rama Bhere & Piyush Kanbhat\n")
```

Append to an existing file (preserves old content)

with open("report.txt", "a") as file:

```
    file.write("Module: T08 — Programming & Automation\n")
```

```
# Write multiple lines at once using writelines()
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

Working with JSON Files

JSON (JavaScript Object Notation) is the most common format for storing and exchanging structured data.

```
import json

# Writing JSON
data = {"name": "Rama", "age": 20, "skills": ["Python", "Logic"]}
with open("profile.json", "w") as f:
    json.dump(data, f, indent=4)

# Reading JSON
with open("profile.json", "r") as f:
    loaded = json.load(f)
print(loaded["name"]) # Rama
```

⚠ Always Use with : Using `open()` without "with" means you must manually call `file.close()`. Forgetting to close a file can corrupt data and exhaust system resources. The "with" statement handles closure automatically, even if an error occurs inside the block.

10. Debugging — Finding and Fixing Problems

Debugging is the systematic process of identifying, analysing, and fixing bugs (errors) in a program. The term originates from 1947 when pioneers of computing literally found a moth inside a relay of the Harvard Mark II computer. Today, debugging is an indispensable skill — experienced developers spend a significant portion of their time debugging, and mastering it separates good programmers from great ones.

The Debugging Mindset

- Do not guess — reproduce the bug reliably first.
- Isolate the problem — narrow down exactly where and when it occurs.
- Understand before fixing — know WHY the bug occurs, not just WHAT it does.
- Fix one thing at a time — changing multiple things makes it impossible to know what worked.
- Verify the fix — confirm the bug is gone without introducing new issues.

Debugging Techniques

Technique 1 : Print Debugging

The simplest technique: insert `print()` statements to inspect variable values at different points in the program. Quick for small scripts but becomes unwieldy in large codebases.

```
def calculate_total(prices):  
    total = 0  
    for price in prices:
```

```
    print(f"DEBUG: price={price}, total before={total}") # Debug line
    total += price
    print(f"DEBUG: total after={total}")                # Debug line
    return total

calculate_total([10, 20, 30])
```

Technique 2 : Using the Python Debugger (pdb)

Python's built-in debugger lets you pause execution and inspect the program state interactively.

```
import pdb

def process_data(data):
    pdb.set_trace()    # Execution pauses here
    result = [x * 2 for x in data]
    return result

# In the pdb prompt:
# n → next line
# s → step into function
# p variable → print variable value
# c → continue running
# q → quit debugger
```

Common Bug Types and How to Fix Them

Bug Type	Symptom	Fix Strategy
Off-by-one error	Loop runs one too many / few times	Check range boundaries carefully
Logic error	Wrong output, no crash	Trace through with print/debugger
None reference	AttributeError on NoneType	Check return values for None
Infinite loop	Program freezes / hangs	Verify loop condition changes
Encoding error	UnicodeDecodeError on file read	Open file with encoding="utf-8"

Using IDE Debuggers

Modern IDEs (VS Code, PyCharm, IntelliJ) include powerful visual debuggers that provide :

- Breakpoints — pause execution at any line by clicking in the gutter.
- Watch expressions — monitor variable values in real time as code runs.
- Call stack inspection — see exactly which functions led to the current point.
- Step Over / Step Into / Step Out — precisely control how you navigate the code.

- Conditional breakpoints — only pause when a specific condition is true.

 **Pro Debugger Habit :** Before reaching for print statements, try rubber duck debugging: explain your code line by line to a rubber duck (or a colleague). Articulating the logic aloud frequently reveals the bug immediately, without writing a single extra line of code.

Quick Reference — All Concepts at a Glance

#	Concept	One-Line Definition	Key Keyword
01	Logic	The rules for making True/False decisions	and, or, not, bool
02	Variables	Named storage containers for data	= (assignment)
03	Data Types	Categories that define what kind of data is stored	int, float, str, bool, list, dict
04	Operators	Symbols that perform operations on data	+, -, *, /, %, ==, !=, >, <
05	Functions	Named, reusable blocks that perform a specific task	def, return, parameters
06	Loops	Constructs that repeat code multiple times	for, while, break, continue
07	Conditions	Structures that choose different paths based on data	if, elif, else

08	Error Handling	Responding to unexpected problems gracefully	try, except, finally, raise
09	File Operations	Reading and writing data to persistent storage	open(), read(), write(), json
10	Debugging	Finding and fixing errors systematically	print(), pdb, IDE debugger